МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине «Аппаратное и программное обеспечение сетей» за 4 семестр

Тема: «Организация UDP – сервера и UDP – клиента»

Вариант 7

Выполнил:
студент 3 курса
группы ИИ-28
(1 подгруппа)
Заремская Е.П.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучить Winsock2 и написать UDP -сервер и UDP -клиент, которые обрабатывают поток символов группами по 64 и рвут связь, если в группе меньше 3 разных символов.

Ход работы

Постановка задачи:

Задания для реализации UDP сервера: после приема каждой группы из 48 символов сервер отсылает подтверждение в виде контрольной суммы ASCII-кодов символов группы и количество символов, принятых сервером от начала сеанса. Если в потоке принятых символов встречается последовательность «~#~», то сервер отсылает сообщение об окончании сеанса и разрывает соединение.

Задания для реализации UDP клиента:

Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: **disconnect <адрес> <порт>**	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: **connect** **<адрес> <порт>** 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
PgUp	1), 3)	-	1)

Код программы:

server.cpp

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <winsock2.h>
#include <windows.h>

#define MY_PORT 666

int main(int argc, char* argv[])
{
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);

    char buff[1024];
    printf("UDP SERVER (TASK 6)\n");

    if (WSAStartup(0x0202, (WSADATA *)&buff[0]))
```

```
    {
        printf("Error WSAStartup %d\n", WSAGetLastError());
        return -1;
    }

    SOCKET my_sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (my_sock == INVALID_SOCKET)
    {
        printf("Socket() error: %d\n", WSAGetLastError());
        WSACleanup();
        return -1;
    }

    sockaddr_in local_addr;
    local_addr.sin_family = AF_INET;
    local_addr.sin_addr.s_addr = INADDR_ANY;
    local_addr.sin_port = htons(MY_PORT);

    if (bind(my_sock, (sockaddr *)&local_addr, sizeof(local_addr)))
    {
        printf("bind error: %d\n", WSAGetLastError());
        closesocket(my_sock);
        WSACleanup();
        return -1;
    }

    printf("Ожидание UDP-пакетов на порту %d\n", MY_PORT);

    char group[48];
    int filled = 0;
    int total = 0;
    int match = 0;
    const char STOP[] = "~#~";
    int hello_sent = 0;

    while (1)
    {
        sockaddr_in client_addr;
        int client_addr_size = sizeof(client_addr);

        int bsize = recvfrom(my_sock, &buff[0], sizeof(buff)-1, 0,
                             (sockaddr *) &client_addr, &client_addr_size);

        if (bsize == SOCKET_ERROR) continue;

        if (!hello_sent)
        {
            const char sHELLO[] = "Hello, Student!\r\n";
            sendto(my_sock, sHELLO, strlen(sHELLO), 0, (sockaddr *)&cli-
ent_addr, client_addr_size);
            printf("+Новый клиент [%s:%d]\n", inet_ntoa(cli-
ent_addr.sin_addr), ntohs(client_addr.sin_port));
            hello_sent = 1;
        }

        for (int i = 0; i < bsize; i++)
```

```
        {
            unsigned char ch = (unsigned char)buff[i];

            if (ch == (unsigned char)STOP[match])      match++;
            else if (ch == (unsigned char)STOP[0])     match = 1;
            else                                       match = 0;

            group[filled++] = (char)ch;
            total++;

            if (match == 3)
            {
                const char *msg = "Сеанс окончен.\r\n";
                sendto(my_sock, msg, strlen(msg), 0, (sockaddr *)&cli-
ent_addr, client_addr_size);
                printf("-Клиент отключен (~#~)\n");

                filled = total = match = 0;
                hello_sent = 0;
                break;
            }

            if (filled == 48)
            {
                int sum = 0;
                char outmsg[128];
                for (int k = 0; k < 48; k++)
                    sum += (unsigned char)group[k];

                sprintf(outmsg, "SUM=%d TOTAL=%d\r\n", sum, total);
                sendto(my_sock, outmsg, strlen(outmsg), 0, (sockaddr *)&cli-
ent_addr, client_addr_size);
                printf("Отправлено: %s", outmsg);
                filled = 0;
            }
        }
    }

    closesocket(my_sock);
    WSACleanup();
    return 0;
}
```

client.cpp

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

#define PORT 666
#define SERVERADDR "127.0.0.1"
```

```
SOCKET my_sock = INVALID_SOCKET;
FILE *g_log = NULL;
int g_connected = 0;
sockaddr_in dest_addr;

void log_event(const char *what, const char *detail)
{
    time_t t;
    struct tm *tm;
    char ts[64];
    if (!g_log) return;
    time(&t);
    tm = localtime(&t);
    sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d",
        tm->tm_year+1900, tm->tm_mon+1, tm->tm_mday,
        tm->tm_hour, tm->tm_min, tm->tm_sec);
    fprintf(g_log, "%s %s %s\n", ts, what, detail ? detail : "");
    fflush(g_log);
}

int main(int argc, char* argv[])
{
    char buff[10240];
    char line[1024];
    int len = 0;
    int c, k;

    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);

    printf("UDP CLIENT (TASK 2)\n");
    printf("Нет автоподключения. Набери: connect 127.0.0.1 666 и нажми
PgUp\n");
    printf("Дальше набор текста, отправка - PgUp, выход - quit + PgUp\n\n");

    g_log = fopen("udp-client.log", "a");

    if (WSAStartup(0x202, (WSADATA *)&buff[0]))
    {
        printf("WSAStart error %d\n", WSAGetLastError());
        return -1;
    }

    my_sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (my_sock == INVALID_SOCKET)
    {
        printf("Socket() error %d\n", WSAGetLastError());
        return -1;
    }

    printf("S<=C:");
    fflush(stdout);

    while (1)
    {
```

```
        c = getch();
        if (c == 0 || c == 224)
        {
            k = getch();
            if (k == 73 && len > 0)
            {
                line[len] = 0;
                if (!strcmp(line, "quit"))
                    break;

                if (!g_connected)
                {
                    char addr[256];
                    int port;
                    if (sscanf(line, "connect %255s %d", addr, &port) == 2)
                    {
                        dest_addr.sin_family = AF_INET;
                        dest_addr.sin_port = htons(port);
                        HOSTENT *hst;

                        if (inet_addr(addr) != INADDR_NONE)
                            dest_addr.sin_addr.s_addr = inet_addr(addr);
                        else if ((hst = gethostbyname(addr)) != NULL)
                            ((unsigned long *)&dest_addr.sin_addr)[0] = ((un-
signed long **)hst->h_addr_list)[0][0];
                        else
                        {
                            printf("\nInvalid address %s\n", addr);
                            len = 0;
                            printf("S<=C:");
                            continue;
                        }

                        sprintf(buff, "%s:%d", addr, port);
                        printf("\nСоединение с %s успешно установлено\n",
buff);
                        log_event("CONNECTION START", buff); // Пункт 1 лога
                        g_connected = 1;
                    }
                    else
                        printf("\nФормат: connect <адрес> <порт>\n");

                    len = 0;
                    printf("S<=C:");
                    fflush(stdout);
                    continue;
                }

                sendto(my_sock, line, len, 0, (sockaddr *)&dest_addr,
sizeof(dest_addr));
                printf("\nотправлено %d симв.\n", len);

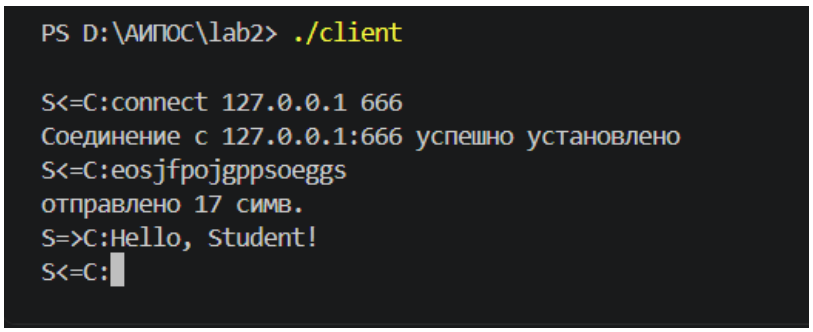
                sockaddr_in from_addr;
                int from_len = sizeof(from_addr);
                int n = recvfrom(my_sock, buff, sizeof(buff)-1, 0, (sockaddr
*)&from_addr, &from_len);
```

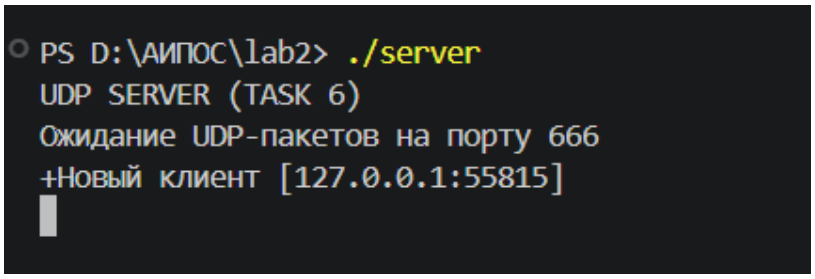
```
                if (n == SOCKET_ERROR)
                    printf("recvfrom() error: %d\n", WSAGetLastError());
                else if (n > 0)
                {
                    buff[n] = 0;
                    printf("S=>C:%s", buff);
                    if (buff[n-1] != '\n') printf("\n");
                    log_event("RECEIVED", buff);
                }

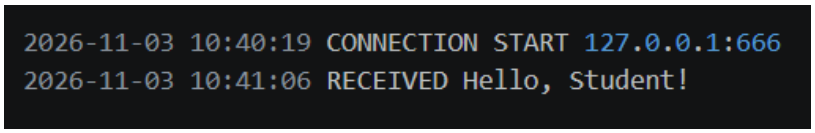
                len = 0;
                printf("S<=C:");
                fflush(stdout);
            }
            continue;
        }
        if (c == 8)
        {
            if (len > 0)
            {
                len--;
                printf("\b \b");
                fflush(stdout);
            }
            continue;
        }
        if (c >= 32 && len < (int)sizeof(line)-1)
        {
            line[len++] = (char)c;
            putchar(c);
            fflush(stdout);
        }
    }

    if (g_connected)
        log_event("CONNECTION END", "");
    if (my_sock != INVALID_SOCKET)
        closesocket(my_sock);
    WSACleanup();
    if (g_log) fclose(g_log);
    return 0;
}
```

Тестирование программы:







Вывод: изучила функции Winsock2 (WSAStartup, socket, bind, sendto, recvfrom, connect, closesocket, WSACleanup); написал и отладил UDP-сервер и клиент для варианта 2; сервер накапливает группы по 48 символов, считает контрольную сумму их ASCII-кодов (SUM) и общее количество символов от начала сеанса (TOTAL), а при встрече ~#~ разрывает сеанс; клиент подключается по команде connect, отправляет строки по PgUp и ведёт лог udp-client.log с фиксацией времени соединения и принятых сообщений; программы проверены, задание выполнено полностью.